Department of Computer Science, University College London

COMP0227 Lab 04: Things in Images: Masks and Boxes

27th October, 2026

1 Scope and Purpose

So far, the maps in these labs have been made of points. Lab 02 gave every pixel a class, such as *car*, but a class does not say which pixels belong to the same car. Lab 02 also mentioned that pySLAM has object detectors, which give each object its own box. This lab uses them.

An object detector gives every object it finds a box, a class and a score, which says how sure the detector is. It first proposes many candidate boxes, and must then keep one box per object. Some detectors only know the classes they were trained on. Others, called *open-vocabulary*, are given their classes as text. A *promptable* segmenter has no classes. Given a point or a box, it draws the *mask* of whatever is there. pySLAM's `semantics` level includes a detector of each kind and a promptable segmenter. With the depth of a frame and the pose of the camera, a detection can be *lifted* into the map as a point in 3D.

This lab has two main purposes. The first is to run these networks on your machines. This gives us a further opportunity to test how pySLAM runs on them. The second is to carry out a small investigation into what a detector finds, and whether its detections can become objects in a map. Specifically:

1. What does a detector find in a single image, and how do its thresholds and its classes change the answer?
2. How good is a detector, measured on images whose objects are known?
3. When the detections of a sequence are lifted into the map, does the map hold the right objects?

You will first run the detectors and the segmenter on single images. You will then write the step which keeps one box per object, and compute the average precision of the detector. Finally you will run SLAM with the detections of a sequence, and look at the objects in the map.

This lab assumes that you have completed Labs 00, 01, 02 and 03, and that pySLAM is installed and working. As before, run all the commands from the `pyslam` folder.

Most runs of this lab take a few seconds. The exceptions are the four SLAM runs of Activity 8, which take a minute or two each. The members of your group can divide them among themselves.

2 Setting Up

Do this before the lab: the downloads are large.

First bring pySLAM up to the version for this lab, as the CHANGELOG describes. From the `COMP0227_26-27` **folder:**

```
git pull
git submodule update --init --depth 1 pyslam
cd pyslam
pixi run build
```

On native Windows the last command is `pixi run -e default-win build`**.** `build` **only does what has changed, and nothing you installed before is lost. The version for this lab is** `comp0227-2026-lab04`**.**

The methods of this lab need the `semantics` **level of pySLAM, which Lab 02 offered for SegFormer. It includes the** `depth` **level of Lab 03. If you do not have it yet, the command below installs it first (about 6 GB), and then downloads the weights of the three networks, 0.35 GB in all, most of it for the text encoder of** `YOLOE`**:**

```
pixi run -e semantics models-detection
```

At the end, the command runs each network once on a test image. It then prints one line for each network. Each of these three lines should begin with `OK`**. If it stops, run it again: whatever is already downloaded is kept.**

- **On Linux without an NVIDIA GPU (in WSL2 too), use** `-e semantics-cpu` **instead of** `-e semantics` **everywhere, and add** `-e default-cpu` **to every** `pixi run` **command of this sheet which has no** `-e`**, straight after** `pixi run`**: for example** `pixi run -e default-cpu slam ...`**.** `docs/PIXI.md` **lists these environments.**
- **If disk space is short,** `pixi clean cache` **deletes the copy of the downloaded packages which pixi keeps.**

Part 2 scores the detector on 59 images of `room0`**, a room of the Replica dataset, whose images are rendered from 3D scans of real rooms. The course repository has the true boxes of the objects in them, the *labelled set*, in the folder** `Labs/Lab_04_-_Things_in_Images/room0`**, which** `git pull` **brought (from the** `pyslam` **folder, it is** `../Labs/Lab_04_-_Things_in_Images/room0`**). It does not have the images: they are read from your own copy of Replica** `room0`**. You may already have it. If not,** `room0` **is one of the eight scenes of the Replica package, to which pySLAM's README gives a link (**`README.md`**, section Replica Datasets; on a Mac, which has no** `wget`**, use** `curl -LO` **with the same address). The package is a single zip file of 12.4 GB, so start the download early, and keep about 15 GB of disk free. Its** `room0` **folder is 1.8 GB: 2000 colour images (**`results/frame000000.jpg` **to** `frame001999.jpg`**, 1.0 GB) and their depth images. This lab reads only the colour images, so one member of your group can download the package and copy the** `room0` **folder, or only its colour images, to the others. The commands below unpack** `room0` **alone (run** `unzip` **in the folder where you saved** `Replica.zip`**; on Linux, install** `unzip` **first if it is missing), set** `R` **to its folder, and, from the** `pyslam` **folder, check that the images are there:**

```
unzip Replica.zip "Replica/room0/*"
R=path/to/Replica/room0
pixi run -e semantics models-detection \
    --set ../Labs/Lab_04_-_Things_in_Images/room0 --images $R
```

(R is just a shorthand for the folder, which can be the room0 folder or the folder of its colour images; Activity 6 uses it too.) The command then prints a fourth line, for the labelled set, which should also begin with OK: it says where it found the 59 images, and warns if they are not the images from which the boxes were made.

Part 3 uses the TUM sequence rgbd_dataset_freiburg1_xyz (fr1_xyz below) of Lab 03. As in Lab 03, S is a shorthand for its folder:

```
S=path/to/rgbd_dataset_freiburg1_xyz
```

Part 3 needs the sequence's associations.txt, which pairs each colour image with the camera's depth image. If you do not have the sequence (0.45 GB), or its folder has no associations.txt, download it and write the file as Lab 03 (Setting Up) describes: without that file, pySLAM reads rgb.txt instead and runs monocular SLAM, without the camera's depth. One variant of Activity 8 also uses the depth which Depth Anything V2 predicted for this sequence in Lab 03 (Activity 4): depth_dav2/ and associations_dav2.txt, in the folder of the sequence. If you do not have them, copy them from a member of your group into your copy of the sequence (0.2 GB). Or predict them again as Lab 03 describes, with the small model (models-depth-small, 0.2 GB of weights, and about 5 minutes on a CPU), for all 792 frames: maps made with --step 5 are not enough for SLAM.

R and S last only until the terminal is closed. On the day of the lab, set them again in each new terminal, with the two lines above. If you forget, a command stops with an error such as expected one argument.

The SLAM commands of Part 3 run in the default environment, as before.

On Native Windows

The semantics level does not exist on native Windows, so object-detection does not run there. Use WSL2 for Parts 1 and 2 and for Activity 7, or work with a member of your group who runs those commands, and answer the questions from their windows and summary lines. Two steps run at the default level, with -e default-win as in Lab 02: the test of your function in Activity 5, and SLAM with the detections in Activity 8. For the second, copy the folder detections/fr1_xyz which a member of your group saved in Activity 7 (about 1 MB) into your pyslam folder, as detections\fr1_xyz. The PowerShell notes of Lab 02 also apply: set the folder with $S = "C:\path\to\rgbd_dataset_freiburg1_xyz", and type each command on one line, without the \ at the ends of the lines.

3 The Methods

Table 1 lists the methods. The names are the ones pySLAM uses.

Name	Network	Gives	Classes
YOLO	YOLO26, small, with masks	a box, a class, a score and a mask for every object it finds	the 80 classes of COCO
YOLOE	YOLOE-26, small, with masks, and a text encoder	the same, for the classes you name	the names you type, one prompt per class
SAM	MobileSAM, a small version of SAM (the Segment Anything Model)	three masks for a point, one for a box	none

Table 1: The methods of this lab. All three need the semantics level. pySLAM has other detectors, which are not used here.

A detector only knows the classes it was trained on. `YOLO` **was trained on COCO (Common Objects in Context), whose 80 classes include person, chair, couch, tv, laptop, keyboard, mouse, cup, bottle and book, but not monitor, desk, lamp or headphones.** `YOLOE` **takes its class names as text, and** `SAM` **has no names at all. The full list of methods, with what each one gives, and COCO's 80 classes are printed by:**

```
pixi run -e semantics object-detection --list
```

4 Part 1: Single Images

Activity 1: Detect the Objects in One Image

Run:

```
pixi run -e semantics object-detection --method YOLO --test desk
```

This finds the objects in `desk`**, the photograph of an office desk which Labs 02 and 03 used (Table 2). The first image takes longer, because the network sets itself up on its first call. Three windows open, perhaps on top of one another (move them apart with the mouse):**

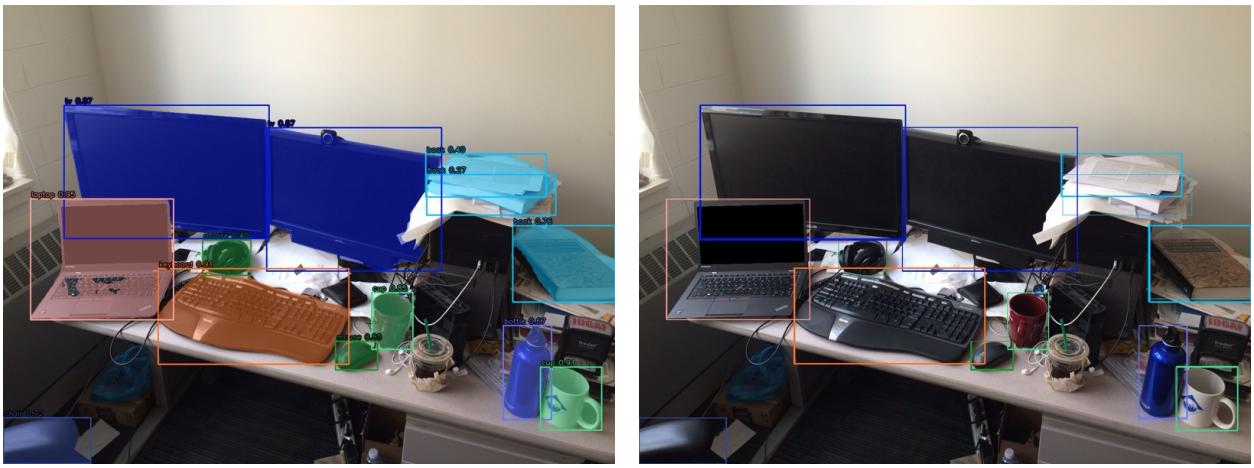
- `Image`**: the image. Its title gives the position of the pixel under the mouse.**
- `Detections`**: every box the detector kept, with its class and its score, and the mask of each object shaded.**
- `Candidates`**: every box whose score was above the score threshold (Activity 2), before the overlapping boxes were removed (Figure 1).**

Press `q` **or Esc in a window to stop. The terminal lists the boxes which were kept, with their corners in pixels, and then a summary:**

```
Summary: YOLO on desk (1280x960 pixels): 80 classes (COCO) on mps; 0.03 s
per image (mean of 3 runs after the first, which took 1.79 s; 0.2 s to load
the model); 13 objects kept of 111 candidates (conf 0.25, iou 0.7): book 3,
cup 2, mouse 2, tv 2, bottle 1, chair 1, keyboard 1, laptop 1
```

Your numbers, especially the times, will be different. `mps` **is the GPU of a Mac; on other computers this reads** `cuda` **or** `cpu`**. On a CPU, one image takes about a tenth of a second, so the runs of Parts 1 and 2 take only seconds there too.** `conf` **and** `iou` **in the summary are the two thresholds of Activity 2.**

- **Which objects did the detector find, and which did it miss? For each one it missed, is its class in COCO at all?**
- **There is one mouse on the desk, and fewer books than the summary says. Find the wrong boxes. What did the detector see in each, and why might it have chosen that class?**
- **Look at the scores. Which object is it least sure of: the smallest, the most hidden, or the one which looks least like its class?**
- **The 111 candidates became 13 objects. Look at the** `Candidates` **window. Where do the extra boxes sit, and why does every object have several?**
- **Run** `--test mars`**. What does the detector find on the surface of Mars, and how sure is it? Unlike the segmentation networks of Lab 02, a detector can return no box at all.**



(a) `Detections` (b) `Candidates`

Figure 1: Two of the windows of Activity 1: the 13 boxes which `YOLO` kept on `desk`, and its 111 candidates.

Activity 2: The Two Thresholds

The detector keeps the boxes whose score is above a score threshold, `--conf` (0.25 by default). Then *non-maximum suppression* (NMS) removes the duplicates, one class at a time. It uses the IoU (intersection over union) of two boxes: the area they share, divided by the area they cover together. NMS keeps the box with the best score, removes every other box of that class whose IoU with it is above `--iou` (0.7 by default), and repeats with the boxes which are left. You will write it in Activity 5. Run the same image with other values:

```
pixi run -e semantics object-detection --method YOLO --test desk --conf 0.05
pixi run -e semantics object-detection --method YOLO --test desk --conf 0.5
pixi run -e semantics object-detection --method YOLO --test desk --iou 0.3
pixi run -e semantics object-detection --method YOLO --test desk --iou 0.95
```

YOLO26 predicts from grids of cells which cover the image: every cell gives a box, a class and a score. The last layers of a network, which make these predictions, are its *head*, and YOLO26 has two heads. The one used so far was trained *one-to-many*: up to ten cells near each object, the object's *positive* cells, learn to predict it. So several of them give a box with a high score on each object, and NMS must remove the duplicates. The other head, the *one-to-one* head, was trained with only one positive cell per object, so it needs no NMS. `--one-to-one` runs it:

```
pixi run -e semantics object-detection --method YOLO --test desk --one-to-one
```

Fill in the table from the summary lines.

	Objects kept
`--conf 0.05`	
`--conf 0.25` **(Activity 1)**	
`--conf 0.5`	
`--iou 0.3`	
`--iou 0.95`	
`--one-to-one`	

- **At `--conf 0.05`, what appears? Are the new boxes on objects, on parts of objects, or on nothing? At `--conf 0.5`, what disappears? Were those the wrong boxes of Activity 1?**
- **At `--iou 0.3`, one box disappears. Which one, and why did it overlap another box of its class so much? At `--iou 0.95`, which objects have two boxes?**
- **Which boxes did the one-to-one head drop, and did it add any? Why does it need no `--iou`?**
- **A robot that must not miss a person and a robot that must not stop for a shadow want different thresholds. Which `--conf` does each want, lower or higher, and what does each lose by that choice?**

Activity 3: Name Your Own Classes

`YOLOE` takes its class names as text. A *prompt* is what you give a network to say what it should find. For `YOLOE`, each prompt is one class name, a word or a short phrase; for `SAM` (Activity 4), it is a point or a box. `YOLOE`'s *text encoder*, a second network, turns each name into a row of the detector's classifier, in the place of a fixed class. `--prompt` takes a list of names, separated by commas:

```
pixi run -e semantics object-detection --method YOLOE --test desk \
    --prompt "monitor, keyboard, mouse, mug, headphones, webcam, lamp, plant"
```

The summary line also says how long loading and running the text encoder took:

```
Summary: YOLOE on desk (1280x960 pixels): 8 prompts on mps; 0.02 s per
image (mean of 3 runs after the first, which took 0.90 s; 0.2 s to load the
model, 2.00 s to load and run the text encoder, once for the list); 7
objects kept of 42 candidates (conf 0.25, iou 0.7): mug 2, headphones 1,
keyboard 1, lamp 1, mouse 1, webcam 1
```

A prompt can also find an object with a score below 0.25, and such a box is not shown: add `--conf 0.05` to see the weaker boxes and their scores.

- **Which of your names found something? Is there a lamp on the desk? Where is the lamp's box?**
- **The monitors were not found. Try `--prompt "tv"`, then `"screen"`, `"display"` and `"computer monitor"`. Which word finds them? Why would the word matter to a network whose classes are not a fixed list?**
- **Do the same for the headphones: `"headphones"`, `"headset"`, `"earphones"`. Then ask, one at a time, for things for which COCO has no class: `"wallet"`, `"computer tower"`, `"desk"`. What is found, and at what score?**
- **Ask for things which are not there, such as `--prompt "cat, bicycle, banana, umbrella"`. Does it find one anyway?**
- **Loading and running the text encoder took longer than the detection. What runs once for each list of words, and what once for each image?**

Take a photograph of your own room with your phone, and copy it to your computer as a JPEG or PNG file (an iPhone saves HEIC files, which OpenCV cannot read: export the photograph as JPEG). Run `YOLO` on it with `--image my/photo.jpg`, and `YOLOE` with `--image my/photo.jpg` and a `--prompt` of your own. Which classes does your room need that COCO lacks? Which words did `YOLOE` need?

Activity 4: A Mask for Whatever You Point At

`SAM` **knows no classes. Give it a point, and it returns the masks of what is there:**

```
pixi run -e semantics object-detection --method SAM --test desk \
    --point 523,651
```

The point (u, v) is in pixels, u from the left and v from the top. This one is on the keyboard. A point does not say how much of an object you mean, so `SAM` **returns three masks, from small to large, each with a score called its *predicted IoU*:** `SAM`**'s own estimate of the IoU between the mask and the true mask of the object. The** `Detections` **window shows them side by side, with their sizes and scores, and the terminal prints a summary:**

```
Summary: SAM on desk (1280x960 pixels) on mps, point (523, 651): 3 masks:
3621 px (score 0.60), 26315 px (0.79), 47422 px (0.93); 0.14 s to encode
the image, 0.14 s for the point (0.2 s to load the model)
```

Your numbers, especially the times, will be different. These are the times of a single call, and on a GPU the first call also sets the network up.

- **What are the three masks: a key, the keys, the keyboard? Which one does** `SAM` **score highest? Is that what you meant?**
- **Point at the headphones (**`--point 467,524`**), at a monitor, and at an edge between two objects (the title of the** `Image` **window gives you the pixel). When is the best-scoring mask not the one you meant?**
- **Give a box instead, with** `--box x1,y1,x2,y2`**: the four numbers of the keyboard's box in the terminal list of Activity 1, separated by commas, without spaces or brackets. Why does a box give one mask, not three?**

A detector's boxes can be given to `SAM`**: the detector names the objects, and** `SAM` **draws their masks.** `--sam` **does this:**

```
pixi run -e semantics object-detection --method YOLOE --test desk \
    --prompt "monitor, keyboard, mouse, mug, headphones, webcam, lamp, plant" \
    --sam
```

- **Compare** `SAM`**'s masks with** `YOLOE`**'s own masks from Activity 3, for the keyboard and the mugs. Which follow the outlines better?**
- **What does** `SAM` **add to the time per image? When would that be worth it?**

Table 2 lists the test images. Use `--test` **to choose one, or** `--image` **with an image file of your own.**

5 Part 2: Measuring a Detector

Activity 5: Write the Suppression Step

In Activity 1, NMS removed most of the detector's candidates. Here you write it: sort the boxes by score, keep the best one, remove every box whose IoU with it is above a threshold, and repeat with the boxes which are left.

Open the file `pyslam/semantics/detection_utils.py`**. It contains the function**

```
def nms(boxes, scores, iou_threshold=0.7):
```

which, as supplied, returns `None`. Its docstring says what it is given and what it must return. In short: it gets an N $\times$ 4 array of boxes (x1, y1, x2, y2, in pixels), an array of N scores and a threshold. It must return the indices of the boxes to keep, best score first. The same file has `box_iou(a, b)`, written for you, which gives the IoU of every box of `a` with every box of `b`, as an array of `len(a)` rows and `len(b)` columns.

Write it. You need about ten lines. Useful functions: `np.argsort(-scores)`, which gives the indices of the boxes from the best score to the worst; `box_iou(boxes[i], boxes[rest])[0]`, where `rest` is an array of the indices of the boxes still left, which gives the IoU of box `i` with each of them; and a comparison such as `iou <= iou_threshold`, which is `True` for the boxes which stay. Test it without any network:

```
pixi run python -m pyslam.semantics.detection_utils
```

This gives your function 30 chair boxes, those which a larger detector found with a score above 0.10 in one of the 59 images of Activity 6. It should reduce them to 3, and the test says whether it did. Then run the detector with your function in place of its own:

```
pixi run -e semantics object-detection --method YOLO --test desk --nms own
```

The script gives your function the candidates of one class at a time, as the detector's own NMS does. The summary line now says how many boxes your function kept, and whether they are those which the detector's own NMS kept. If your function still returns `None`, a warning says so, and the detector's own NMS is used.

- **Do the two agree? Try `--iou 0.3` and `--iou 0.95` as well, where Activity 2 kept 12 and 16 boxes.**
- **With `--nms own --all-classes`, your function gets the candidates of all the classes at once: this is *class-agnostic* NMS. Does it change anything on `desk`? What could it change when two classes compete for one object, such as a lamp shade of `room0` (Activity 6) detected both as a sink and as a bowl?**
- **Two chairs stand side by side and overlap in the image. What does NMS at `--iou 0.5` do to them? Is there a threshold which keeps both without keeping duplicates elsewhere?**

Activity 6: Score the Detector

The folder `../Labs/Lab_04_-_Things_in_Images/room0` holds the 351 true boxes of the chairs, couches, tables, vases and books in 59 images of Replica `room0`: the frames which a SLAM run on `room0` chose as

Name	The image
`kitti`	a frame of the KITTI 06 video (a street)
`kitti_next`	the frame half a second later
`desk`	a desk with objects on it (indoors)
`churchill`	a building
`box`	a box among other objects
`mars`	the surface of Mars

Table 2: The test images (the same as in Labs 02 and 03).

keyframes. The images themselves are those of your copy of `room0` (Setting Up). We made the boxes from the class which the dataset gives every pixel, so touching objects of one class share a box, and an object cut in two by something in front of it has two. Score the detector on them:

```
pixi run -e semantics object-detection --method YOLO \
    --evaluate ../Labs/Lab_04_-_Things_in_Images/room0 --images $R
```

For each class, this puts the detections of all 59 images in one list, ranked by score from the highest down. Then it goes down the list, one detection at a time:

- **The detection takes the true box of its class which it overlaps most, among the true boxes not yet taken. If their IoU is at least 0.5, the detection is a *true positive*; otherwise it is a *false positive*.**
- **After each detection, the *precision* is the share of the detections so far which are true positives, and the *recall* is the share of all the true boxes which have been found so far.**

Going down the list, the recall grows, and the pairs of recall and precision draw the *precision-recall curve* of the class. The summary line gives three numbers, computed from these curves as the COCO benchmark computes them:

- **AP50, the *average precision* at an IoU of 0.5: the area under the curve. At each of 101 recall values, 0, 0.01, . . . , 1, it takes the best precision reached at that recall or at any higher recall, and averages these 101 values. At most 100 detections of each class are scored in each image.**
- **mAP50: the mean of AP50 over the five classes.**
- **mAP50:95: a stricter measure. For each class, AP is computed at ten IoU thresholds, 0.50, 0.55, . . . , 0.95, and the ten values are averaged; then the mean is taken over the five classes.**

AP is computed from every box with a score above 0.001 (the summary's `conf 0.001`), not only from those above 0.25:

```
Summary: YOLO on room0 (59 images, 351 true boxes, 5 classes) on mps, conf
0.001, iou 0.7: AP50 chair 0.93, couch 0.24, dining table 0.22, vase 1.00,
book 0.29; mAP50 0.54, mAP50:95 0.43; 0.05 s per image
```

The `Detections` window shows each image as it is scored, with the boxes whose score is at least 0.25, and the true boxes dashed (Figure 2): `p` pauses, `n` then steps to the next image, and `p` again continues. `q` stops the run early, but then the true boxes of the images not yet seen count as missed, and the summary says so. At the end, a `Precision-recall` window draws the curve of each class; `q` or Esc closes the windows. A run takes about ten seconds, on a GPU or on a fast CPU.

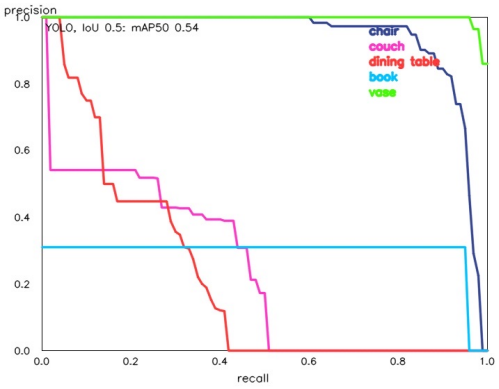
For the last row of the table, `YOLOE` is told the names of the five classes:

```
pixi run -e semantics object-detection --method YOLOE \
    --evaluate ../Labs/Lab_04_-_Things_in_Images/room0 --images $R \
    --prompt "chair, couch, dining table, vase, book"
```

Fill in the table from the summary lines. The first row is the first command above, and each of the next five adds its options to it. To save time, the members of your group can divide the rows among themselves.



(a) `Detections`, on one of the 59 images



(b) `Precision-recall`, at the end

Figure 2: The windows of Activity 6's first command. Left: the detections with a score of at least 0.25 (solid) and the true boxes (dashed); a true box with no detection on it is missed. Right: the precision-recall curve of each class over the 59 images.

	mAP50	mAP50:95	AP50 chair
`YOLO`			
`--conf 0.25`			
`--nms own`			
`--nms own --all-classes`			
`--nms none`			
`--one-to-one`			
`YOLOE`, **the five names**			

- **Which class has the best AP50, and which the worst? Look at the worst class's images in the `Detections` window. Is the detector wrong, or the true box?**
- **With `--conf 0.25`, AP falls. Why, when AP already takes the scores of the boxes into account?**
- **With `--nms own`, the three numbers should be those of the first row, to the last digit. Are they? Check that the summary says `your nms()`: if it does not, your function returned `None`. With `--all-classes`, AP falls. What does one call for all the classes remove that a call per class keeps?**
- **With `--nms none`, every candidate is kept. Why does a duplicate box lower AP so much?**
- **With `--one-to-one` there is no NMS, yet AP is nearly as high. Why?**
- **Why might `YOLOE` get a lower AP50 on the chairs than a detector trained on COCO's chairs? Look at the chairs of this room in the `Detections` window: what kind of seats are they? An ottoman or a pouf is a footstool, a low padded seat. Run the `YOLOE` command again with three more prompts inside the quotes, after `book`: `pouf=chair, ottoman=chair, stool=chair`. `YOLOE` looks for the word on the left of each `=`, and its boxes count as the class on the right. What changes?**
- **Fifty-nine images of one room are a small test. What would you need before you believed these numbers for a robot in another building?**

6 Part 3: Objects in the Map

Activity 7: Detect the Objects in a Sequence

The same script finds the objects in every frame of a sequence, and can save the detections for SLAM:

```
pixi run -e semantics object-detection --method YOLO --tum $S \
    --settings settings/TUM1.yaml --save detections/fr1_xyz --step 5
```

`--step 5` **detects every fifth frame of the 792: about ten seconds on a GPU, and under a minute on a CPU. The** `Detections` **window shows the frames as they are detected;** `p` **pauses and resumes,** `n` **steps while paused, and** `q` **or Esc stops. The folder** `detections/fr1_xyz`**, in the** `pyslam` **folder, gets** `detections.csv`**, with one line per detection (its frame, class, score and box), the masks of each frame in** `masks/`**, and** `counts.csv`**, with one line for each detected frame: its number, its timestamp, the time it took, and the number of objects of each class.**

```
Summary: YOLO on rgbd_dataset_freiburg1_xyz (159 of 792 frames, every 5,
640x480) on mps: 0.03 s per frame; 1297 detections (conf 0.25); per frame:
book 1.7, tv 1.6, chair 1.1, cup 1.0, keyboard 0.9, laptop 0.7, person 0.6,
mouse 0.5; 159 frames saved to detections/fr1_xyz
```

There is one keyboard on the desk. To see in how many frames the detector finds exactly one, plot the number of keyboards in each frame against the frame number, with the numbers of three other classes. Open `counts.csv` **in a spreadsheet, or in Python:**

```
import pandas as pd
import matplotlib.pyplot as plt
counts = pd.read_csv("detections/fr1_xyz/counts.csv")
counts.plot(x="frame", y=["keyboard", "tv", "cup", "person"])
plt.show()
```

(`pixi run python` **opens a Python with pandas.)**

- **Watch the monitor, the keyboard and the mug as the camera moves (the run is short: press** `p` **to pause it, or run it again). Are they found in every frame? Does a box change its class from one frame to the next?**
- **In how many frames is the keyboard's count exactly 1? What do the frames with 0 or 2 show?**
- **Someone sits at another desk, at the back of the room. Which boxes and classes does the detector give them?**

Activity 8: Lift the Detections into the Map

With the depth of a frame and the pose of its camera, a detection becomes a point in the map. pySLAM takes the *centroid* of the detection's mask, the mean position of its pixels. It reads the median depth under the mask (at least 50 of its pixels must have a depth), lifts the centroid to 3D with that depth and the camera's calibration, and moves the point into the map with the pose of the frame (Figure 3). It then joins the points into objects: in order of score, each point joins the nearest object of its class whose centre, the mean of the object's points, is at most 0.35 m away, and otherwise starts a new object. Run SLAM with the saved detections:

```
pixi run slam --tum $S --settings settings/TUM1.yaml \
    --detections detections/fr1_xyz
```

As in Lab 03, this is RGB-D SLAM with the camera's depth, from `associations.txt`. Once SLAM has the pose of a frame which has detections, it lifts those whose score is at least 0.5. The `Map Viewer` draws each lifted detection as a small point, and each object merged so far as a large one. Each class has the same colour as in the `Detections` window of Activity 7.

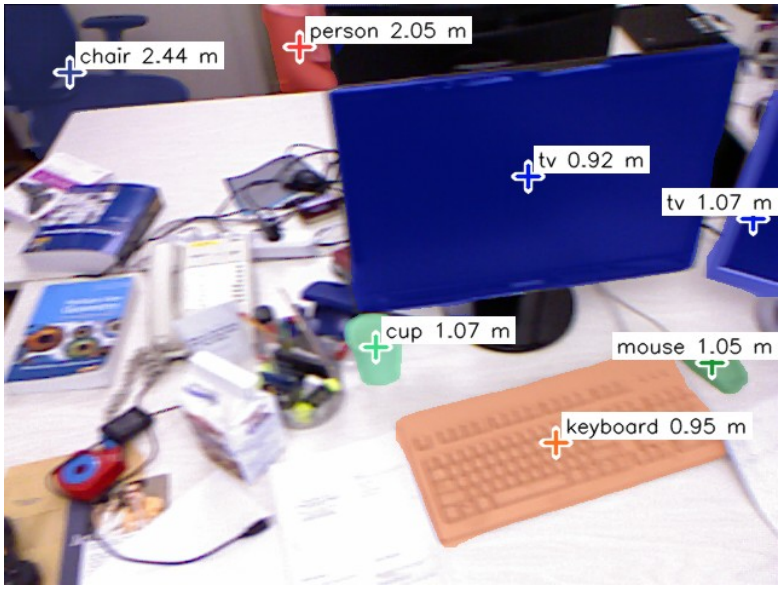


Figure 3: One frame of `fr1_xyz` as pySLAM lifts it: the masks of its detections with a score of at least 0.5, the centroid of each mask (a cross), and the median depth under the mask.

The run takes a minute or two. While it runs, watch the objects appear in the `Map Viewer`, and go back to the last question of Activity 7. pySLAM has a known fault which now and then freezes a run: the frames stop advancing, and the run uses no CPU. Ctrl-C does not stop it. Close the terminal window (on Linux and macOS, Ctrl-\ also works), open a new one, set `S` again, and run the command again.

At the end of the sequence the windows stay open. Look at the objects in the `Map Viewer` now (Figure 4 shows those of our run, from above): when you press `q` or Esc in the `Camera` window, all the windows close. pySLAM then lifts the detections again, with the poses of the final trajectory, and merges them into objects. After the trajectory error, the terminal lists the objects, one line for each, with its class, its number of detections and its *spread*: the largest distance of one of its points from the object's centre. The list ends with a line which counts the objects of each class. Its last number counts the objects made of a single detection, which are among those counted before:

```
Objects: 27 from 884 detections in 158 frames (score at least 0.5; the same
class within 0.35 m): laptop x6, tv x6, chair x4, book x3, keyboard x2,
mouse x2, person x2, cup x1, teddy bear x1; 5 single detections
```

Your numbers will be different. As you saw in Lab 01, two SLAM runs with the same settings do not give quite the same result, so do not read much into a difference of one or two objects between two runs. To save time, the members of your group can divide the variants in the questions below among themselves. The member who predicted the depth of `fr1_xyz` in Lab 03 can run the one with `associations_dav2.txt`.

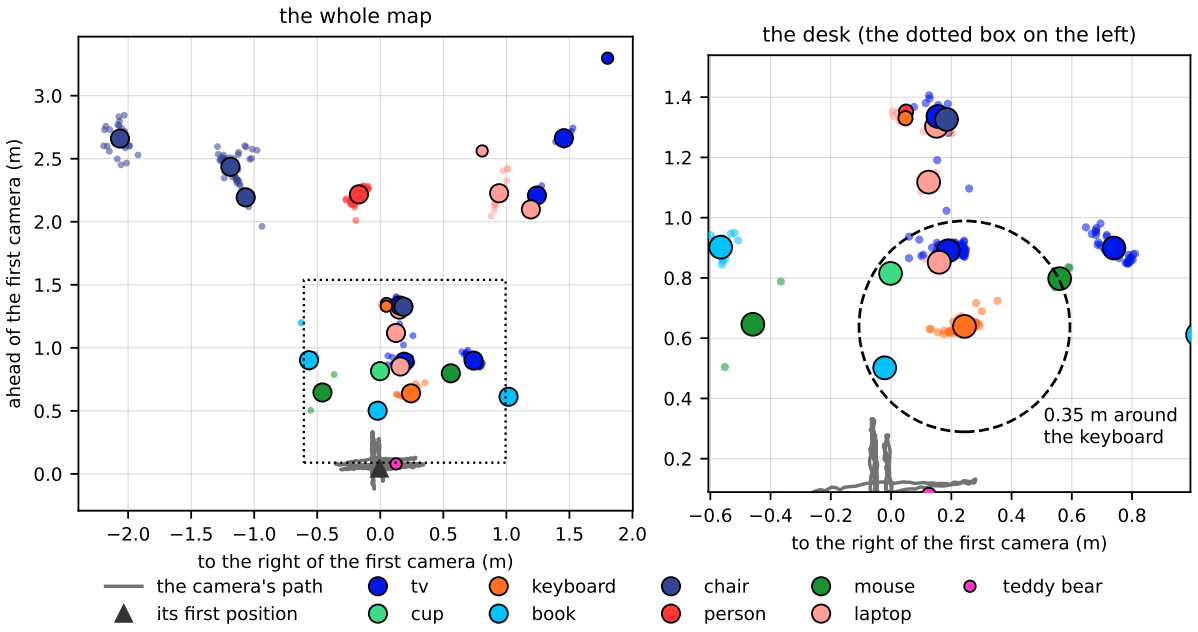


Figure 4: The map of the first command of Activity 8, seen from above: the path of the camera, every lifted detection (small points) and the 27 objects (large points), in the colours of their classes, with the merge radius drawn around the keyboard. The map is that of our run, so yours will differ a little. Its axes are those of the first camera, which looked down at the desk, turned so that the view is from straight above.

- **Does the map have the right number of keyboards, mice, mugs (COCO's `cup`) and monitors (COCO's `tv`)? Find where the extra objects are in the `Map Viewer`. Is one of them the same thing under two names?**
- **Run it with `--merge-radius 0.15`. How many objects are there now? Why would a monitor need a larger radius than a mug?**
- **Run it with `--lift-depth centre`, which reads the depth at the centre of the box instead of the median under the mask. Which objects change: the mug, or the chairs, whose centre pixel may lie on what is behind them?**
- **Add `--associations associations_dav2.txt`, to run it with the depth predicted in Lab 03 in place of the camera's depth (*pseudo RGB-D*, as in Lab 03, Part 3). Compare the mug, the keyboard and the mouse, and their spreads, with those from the camera's depth. Why do they spread more? The whole map is now built from the network's depth. From Lab 03 (Activity 4) you know that the median scale of this network on this sequence, the camera's depth divided by the predicted depth, is about 0.8: what does that do to the size of the map, and to the distances between objects?**
- **The person sat still. Should they be an object in the map?**
- **The objects are points with a class and a count. What else would a robot need to know about them to pick one up, or to walk around it?**

7 Extra Activities

These are optional.

Activity 9: Your Own Room

If you recorded a video of a room in Lab 01 (Activity 7) and made a settings file for your camera, find the objects in it and lift them into the map:

```
pixi run -e semantics object-detection --method YOLO --video my/video.mp4 \
    --save detections/myroom --step 5
pixi run slam --video my/video.mp4 --settings settings/MY_CAMERA.yaml \
    --detections detections/myroom
```

This is monocular SLAM, with no depth image. pySLAM reads the depth of a detection from the map points it tracks inside the mask (at least three of them), so the positions, the merge radius and the spreads are in the units of the map, not in metres; the run says so. If you have no video of your own, use the KITTI 06 video which comes with pySLAM, `data/videos/kitti06/video_color.mp4`**, with** `settings/KITTI04-12.yaml`**: a street, not a room, but the same questions apply.**

- **Which of your objects end up in the map, and which do not? Why?**
- **Is 0.35 still a sensible merge radius in these units? How could you find the scale of your map?**